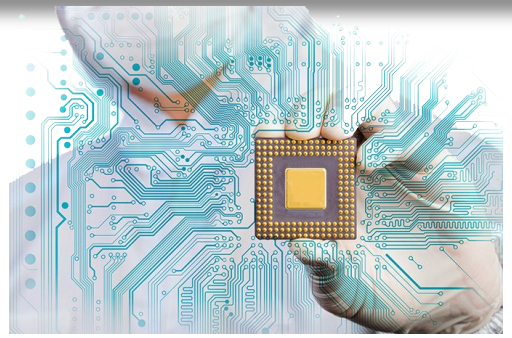
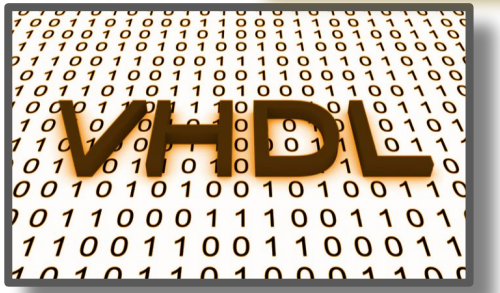
SENTENCIAS CONCURRENTES EN VHDL

PRINCIPIO DE DISEÑO VLSI
DISEÑO DIGITAL CON TÉCNICAS DE ALTO NIVEL
ÁREA DISPOSITIVOS
DEPTO. DE ELECTRÓNICA Y COMPUTACIÓN
FI.UNMdP



SENTENCIAS CONCURRENTES

- SENTENCIA PROCESS
- ASIGNACIÓN A SEÑAL CONCURRENTE
- ASIGNACIÓN CONCURRENTE CONDICIONAL
- ASIGNACIÓN CONCURRENTE CON SELECCIÓN
- LLAMADA CONCURRENTE A SUBPROGRAMA
- COMPONENTES
- SENTENCIA GENERATE
- CONFIGURACIÓN DE UN DISEÑO
- GENÉRICOS



SON SENTENCIAS QUE SE EJECUTAN EN PARALELO
ALGUNAS SENTENCIAS CONCURRENTES TIENEN SU EQUIVALENTE EN LAS SENTENCIAS SECUENCIALES

SENTENCIA PROCESS

PROCESO: SENTENCIA CONCURRENTE QUE CONTIENE SENTENCIAS SECUENCIALES.

SINTAXIS:

```
[etiqueta:] process [(nombre_señal{ ,... })] [is]
        declaraciones
    begin
        sentencias_secuenciales;
end process [etiqueta];
```

- En “declaraciones” se definen valores locales al proceso, o subprogramas locales al mismo.
- Se trata de un lazo infinito, al llegar al final vuelve a la primera sentencia secuencial. La ejecución sólo se detiene si se encuentra la sentencia wait.

SENTENCIA PROCESS

FORMA SIMPLIFICADA DE ESCRIBIR UN PROCESO (SIN WAIT)

```
process
begin
    sentencias_secuenciales;
    wait on lista de sensibilidad;
end process;
```

```
process (lista_de_sensibilidad)
begin
    sentencias_secuenciales;
end process;
```

LA LISTA DE SENSIBILIDAD DEBERÍA CONTENER TODAS LAS SEÑALES QUE SON LEÍDAS POR EL PROCESO

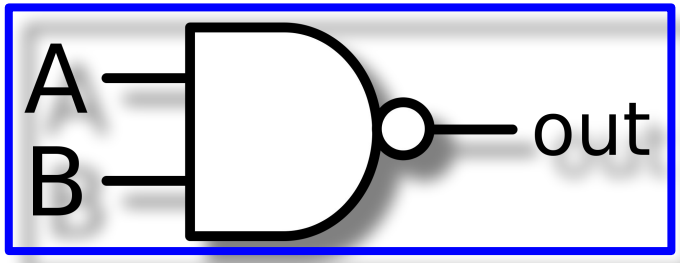
```
process (a, b, c)
begin
   y <= a or b or c;
end process;
```

```
process (a)
begin
   y <= a or b or c;
end process;
```

SENTENCIA PROCESS

EJEMPLO
MODELO DE COMPUERTA NAND DE 2 ENTRADAS

```
library ieee;
    use ieee.std_logic_1164.all;

entity nand2 is
    port( A, B : in std_logic;
              c : out std_logic);
end nand2;
```




```
architecture uno of nand2 is
begin
    process(A,B)
        variable temp : std_logic;
    begin
        temp := not(A and B);
        if (temp = '1') then
            c <= temp after 6 ns;
        elsif (temp = '0') then
            c <= temp after 5 ns;
        else
            c <= temp after 6 ns;
        end if;
    end process;
end nand2;
```

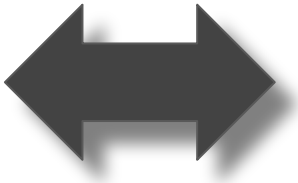
ASIGNACIÓN A SEÑAL CONCURRENTE

LA ASIGNACIÓN A SEÑAL CONCURRENTE SE PUEDE DAR FUERA DE UN PROCESO, EN LA ARQUITECTURA.

SINTAXIS:

```
[etiqueta:] nombre_señal <= [transport] forma_onda;
```

```
a <= b;
```



```
process (b)
begin
a <= b;
end process;
```

LA ASIGNACIÓN CONCURRENTE ES SENSIBLE A TODAS LAS SEÑALES QUE SE ENCUENTRAN A LA DERECHA DE LA ASIGNACIÓN.

ASIGNACIÓN A SEÑAL CONCURRENTE CONDICIONAL

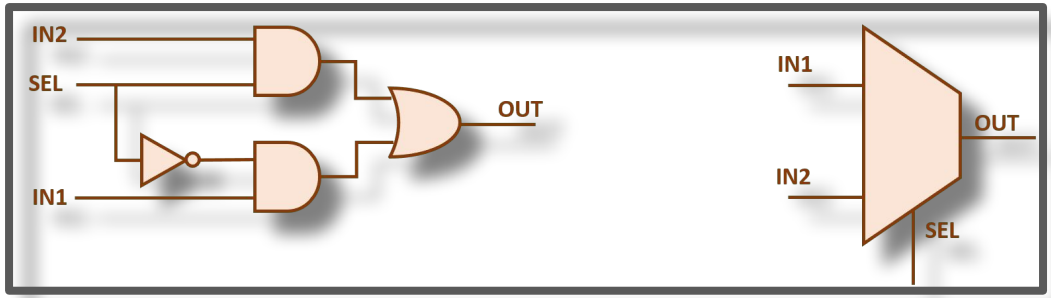
ES UNA FORMA COMPACTA DE LA SENTENCIA IF… ELSIF… ELSE.

SINTAXIS:

```
[etiqueta:] nombre_señal <= [transport]
                    {forma_onda when expresión_booleana else }
                     forma_onda [when expresión_booleana];
```

EJEMPLO

```
MULTIPLEXOR:
    salida <= IN1 when SEL = '0' else
                IN2;
```




```
process (SEL, IN1, IN2)
begin
    if SEL = '0' then
        salida <= IN1;
    else
        salida <= IN2;
    end if;
end process;
```

ASIGNACIÓN A SEÑAL CONCURRENTE CON SELECCIÓN

ES UNA FORMA COMPACTA DE LA SENTENCIA CASE.

SINTAXIS:

```
[etiqueta:] with expresión select
        nombre_señal <= [ transport ]
        {forma_onda when valor,}
         forma_onda when valor,
         unaffected when valor,
         forma_onda when others;
```

UNIDAD ARITMÉTICA-LÓGICA CON DISTINTAS OPERACIONES:

```
with Oper select
 Result <= Op1 + Op2 when suma,
           Op1 - Op2 when resta,
           Op1 and Op2 when andl,
           Op1 or Op2 when orl;
```

```
process(Op1, Op2, Oper)
begin
    case Oper is
        when suma =>
            Result <= Op1 + Op2;
        when resta =>
            Result <= Op1 - Op2;
        when andl =>
            Result <= Op1 and Op2;
        when orl =>
            Result <= Op1 or Op2;
    end case;
end process;
```

LLAMADA CONCURRENTE A SUBPROGRAMA

LA LLAMADA A SUBPROGRAMA PUEDE EXISTIR EN UNA ARQUITECTURA FUERA DE CUALQUIER PROCESO.

SE TRATA DE UNA LLAMADA CONCURRENTE, SE EJECUTA CADA VEZ QUE SE PRODUZCA UN EVENTO EN ALGUNO DE SUS PARÁMETROS.

SINTAXIS:

- Llamada a procedimiento:

```
[etiqueta:] nombre_procedimiento[{parámetros}];
```

- Llamada a función:

```
nombre_función[{parámetros}];
```

SENTENCIAS ESTRUCTURALES, COMPONENTES

PARA PODER HACER USO DE UN COMPONENTE PRIMERO SE LO DEBE DECLARAR. LA DECLARACIÓN PUEDE APARECER EN LA ARQUITECTURA O EN UN PAQUETE.

DECLARACIÓN, SINTAXIS:

```
component nombre_componente [is]
    [generic( lista_generic );]
    [port( lista_puertos );]
end component [nombre_componente];
```

LA DECLARACIÓN DE UN COMPONENTE COINCIDE CON SU ENTIDAD (MISMOS PUERTOS, DIRECCIÓN, ETC.).

LA COPIA DE UN COMPONENTE SE EJECUTA CADA VEZ QUE OCURRE UN EVENTO EN ALGUNA DE LAS SEÑALES CONECTADAS A SUS PUERTOS DE ENTRADA O DE ENTRADA-SALIDA.

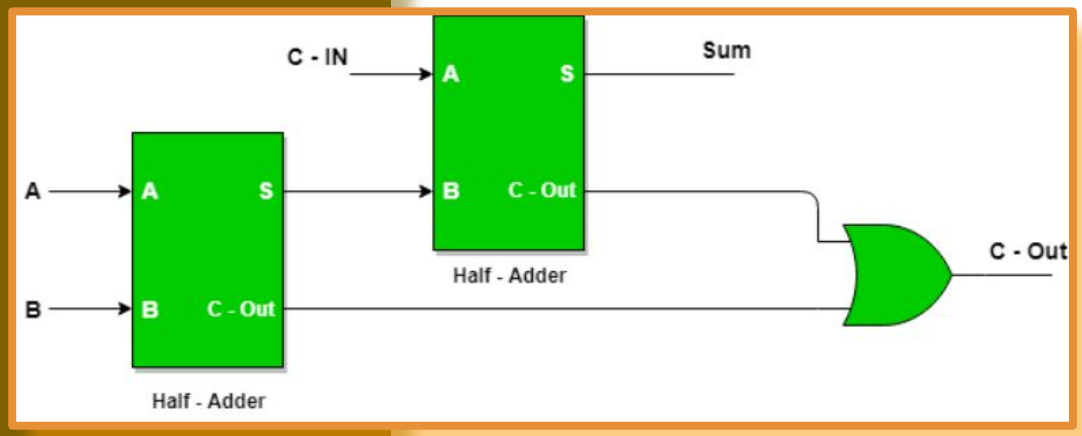
REFERENCIA, SINTAXIS:

```
etiqueta_referencia : nombre_componente
    [generic map (lista_asociación);]
    [port map (lista_asociación);]
```

SENTENCIAS ESTRUCTURALES, COMPONENTES

EJEMPLO: SUMADOR TOTAL A PARTIR DE DOS SEMISUMADORES

```
entity FULL_ADDER is
     port ( X, Y, Cin : in bit;
            Cout, Sum : out bit; );
end FULL_ADDER;

architecture UNO of FULL_ADDER is

   component HALF_ADDER
      port ( I1, I2 : in bit;
          Cout, Sum : out bit );
   end component;

   component PuertaOr
      port ( I1, I2 : in bit;
                  O : out bit);
   end component;

   signal A, B, C : bit;

begin
   U1 : HALF_ADDER port map (X, Y, A, B);
   U2 : HALF_ADDER port map (B, Cin, C, Sum);
   U3 : PuertaOr port map (A, C, Cout);
end UNO;
```



HAY DOS FORMAS DE ASOCIAR LOS PUERTOS:

- En el mismo orden en que aparecen en la declaración del componente (ejemplo anterior).

```
U1 : HALF_ADDER port map (X, Y, A, B);
```

- Forma explícita, no importa el orden:

```
U2 : HALF_ADDER port map (I1 => B, I2 =>
Cin, Cout => C, Sum => Sum );
```

CON CUALQUIERA DE LAS DOS OPCIONES SE PUEDEN DEJAR PUERTOS DE SALIDA SIN CONEXIÓN MEDIANTE LA PALABRA RESERVADA OPEN.

SENTENCIA GENERATE

PERMITE GENERAR MÚLTIPLES COPIAS, TOTALES O PARCIALES, DE ELEMENTOS IGUALES.

- SINTAXIS:

```
etiqueta_generate : {[for especificación_for | if condición]}
    generate [{parte_declarativa} begin]
    {sentencias concurrentes}
end generate;
```

LA ETIQUETA ES OBLIGATORIA.

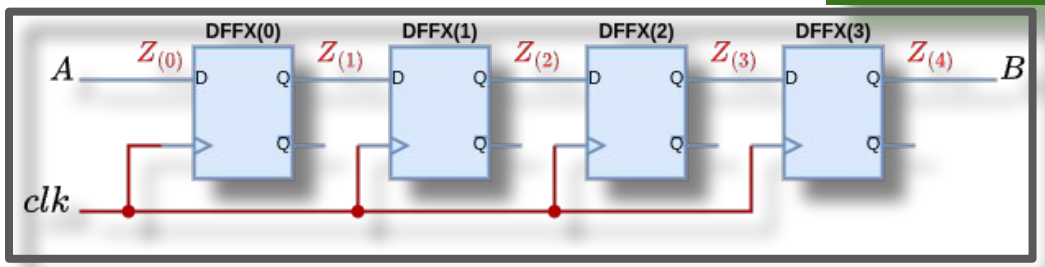
PUEDE CONTENER CUALQUIER SENTENCIA CONCURRENTE, INCLUSO OTRA SENTENCIA GENERATE.

SENTENCIA GENERATE

EJEMPLO

```
REGISTRO DE DESPLAZAMIENTO DE 4 BITS
library ieee;
     use ieee.std_logic_1164.all;
entity shift is
     port( A, clk : in std_logic;
                B : out std_logic);
end shift;
```

```
architecture gen_shift of shift is
component dff
     port(d, clk : in std_logic;
                q : out std_logic);
end component;
     signal z : std_logic_vector(0 to 4);
begin
   z(0) <= A;
   g1 : for i in 0 to 3 generate
         dffx : dff port map(z(i),clk,z(i + 1));
   end generate;
   B <= z(4);
end gen_shift;
```




```
architecture desplazamiento of shift is
     component dff
          port( d, clk : in std_logic;
          q : out std_logic);
     end component;

signal z : std_logic_vector( 0 to 4 );
```

```
begin
     z(0) <= a;
     dff1: dff port map( z(0), clk, z(1) );
     dff2: dff port map( z(1), clk, z(2) );
     dff3: dff port map( z(2), clk, z(3) );
     dff4: dff port map( z(3), clk, z(4) );
     b <= z(4);
end desplazamiento ;
```

SENTENCIA GENERIC

PERMITE EL DISEÑO PARAMÉTRICO. CUANDO EL MODELO SE USA COMO UN COMPONENTE EL USUARIO FIJA EL VALOR QUE DEBEN TOMAR LOS GENÉRICOS DE FORMA DE ADAPTARLO A SUS NECESIDADES. EJEMPLO, COMPUERTA OR DE N ENTRADAS:

```
entity orn is
  generic ( RetInt : time := 1 ns; RetCar : time := 0.2 ns; Carga : real := 0.25 );
  port ( Entradas : in bit_vector; Salida : out bit );
end orn;

architecture Funcional of orn is
begin
   process (Entradas)
        variable Resultado : bit := '0';
   begin
        for i in Entradas'range loop
             if Entradas(i) = '1' then
                  Resultado := '1';
                  exit;
             end if;
        end loop;
        Salida <= Resultado after ( RetIner + ( RetCar * Carga ));
   end process;
end Funcional;
```

¡El circuito final se sintetiza al momento de compilar el circuito en el cual este componente es invocado!

SENTENCIA GENERIC

UNA VEZ DEFINIDO EL MODELO SE LO PUEDE REFERENCIAR:

```
PuertaOr : orn generic map (2 ns, 0.4 ns, 0.5) port map (VectEnt, Salida);
```

LAS REGLAS QUE SE SIGUEN EN GENERIC MAP SON:

- Los valores que se asignan a cada genérico deben ser constantes o expresiones.
- Puede usarse asociación posicional o por nombre o una mezcla de ambas (iguales que las cláusulas port map).
- Se puede omitir el valor a asignar a cualquier genérico si éste tiene valor predefinido.
- Puede explicitarse que quiere usarse el valor por defecto si se asigna la palabra reservada open.

LA SIGUIENTE REFERENCIA ES EQUIVALENTE A LA ANTERIOR:

```
PuertaOr : orn generic map (RetIner => 2 ns, RetCar => 0.4 ns, CargA => 0.5)
                    port map (VectEnt, Salida);
```

LAS SIGUIENTES COPIAS DE COMPONENTES SON TODAS EQUIVALENTES:

```
PuertaOr : or_n port map (VectEnt, Salida);
PuertaOr : or_n generic map (open, o[illegible]) port map (VectEnt, Salida);
PuertaOr : or_n generic map (1 ns [illegible] port map (VectEnt, Salida);
```